

# Playwright Stabilization Assessment Report

**Submitted By**

**Name:** Saiteja Kodi

**Technology:** Playwright with TypeScript

**Repository:** [ust-sdet-training/SDET at saiteja\\_week05\\_gate5](#)

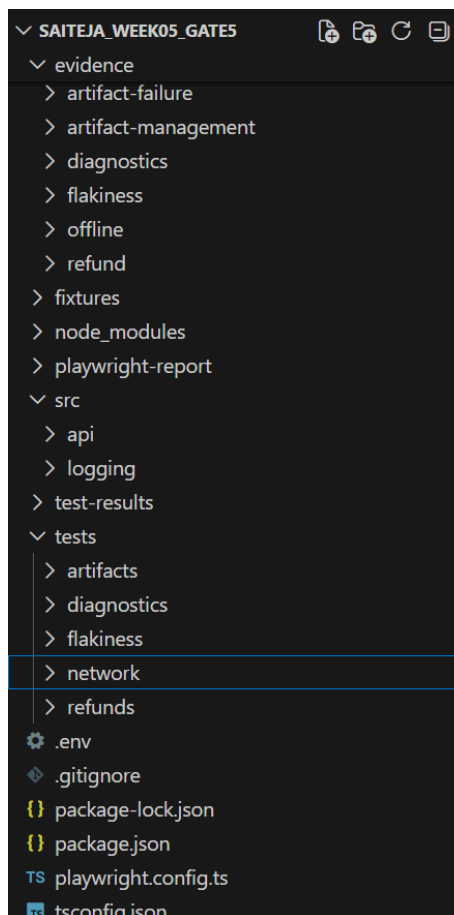
---

## Note:

Meaningful comments have been added throughout the test files to improve readability and make the test flow easier to understand. This report presents each assessment scenario using the **Diagnose → Fix → Evidence → Explain** approach, supported by relevant screenshots.

---

## Code Structure:



## 1. Objective

The objective of this assessment is to stabilize the Playwright automation suite by identifying failures, diagnosing their root causes, validating fixes, and providing supporting evidence. The assessment focuses on writing reliable tests and demonstrating debugging maturity.

---

## 2. Assessment Overview

This assessment covers the following topics:

- Flakiness Analysis
- Diagnostic Logging
- Artifact Management
- Offline & Resilient UI
- Returns & Refund Testing

The overall approach followed throughout the assessment is:

**Diagnose → Fix → Evidence → Explain**

---

## 3. Tools Used

- Playwright
  - TypeScript
  - Node.js
  - Visual Studio Code
  - Git & GitHub
- 

## 4. Flakiness Analysis

**Command to run in terminal:**

```
npx playwright test makeltFlaky.spec.ts --repeat-each=50
```

**Test File:** makeltFlaky.spec.ts

**Scenario 1 – Race Condition**

## Issue

The cart total was verified before the asynchronous request completed.

## Diagnosis

The assertion executed before the UI finished updating, causing intermittent failures.

## Fix

Use Playwright's web-first assertions and wait for the UI state instead of using very short timeouts.

## Evidence

- Screenshot of failed test (Before)

```
15     await page.getByRole("button", { name: "Refresh cart total" }).click();
16
17     // Check the cart total immediately, which is the flaky part.
18     await expect(page.getByTestId("debug-cart-total")).toContainText("Rs. 9,197", {
19         timeout: 620
20     });
21 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT

before the async request has settled

4 flaky

[chromium] > tests\flakiness\makeItFlaky.spec.ts:10:7 > Intentionally flaky example > bad example: asserts cart total before the async request has settled

[chromium] > tests\flakiness\makeItFlaky.spec.ts:10:7 > Intentionally flaky example > bad example: asserts cart total before the async request has settled

[chromium] > tests\flakiness\makeItFlaky.spec.ts:10:7 > Intentionally flaky example > bad example: asserts cart total before the async request has settled

[chromium] > tests\flakiness\makeItFlaky.spec.ts:10:7 > Intentionally flaky example > bad example: asserts cart total before the async request has settled

13 passed (1.6m)

Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.

- Screenshot of stable execution (After)

```
16
17 // Check the cart total immediately, which is the flaky part.
18 await expect(page.getByTestId("debug-cart-total")).toContainText("Rs. 9,197");
19 });
20
```

```
C:\Users\299172\Documents\Saiteja_week05_gate5>npx playwright test makeItFlaky.spec.ts

Running 2 tests using 2 workers
  2 passed (10.2s)

To open last HTML report run:

  npx playwright show-report
```

- HTML Report screenshot (Before)

|                                                                                                      |            |       |
|------------------------------------------------------------------------------------------------------|------------|-------|
| ⚠ Intentionally flaky example › bad example: asserts cart total before the async request has settled | chromium   | 13.0s |
| flakiness/makeltFlaky.spec.ts:10                                                                     | View Trace |       |
| ⚠ Intentionally flaky example › bad example: asserts cart total before the async request has settled | chromium   | 15.7s |
| flakiness/makeltFlaky.spec.ts:10                                                                     | View Trace |       |
| ⚠ Intentionally flaky example › bad example: asserts cart total before the async request has settled | chromium   | 10.1s |
| flakiness/makeltFlaky.spec.ts:10                                                                     | View Trace |       |
| ⚠ Intentionally flaky example › bad example: asserts cart total before the async request has settled | chromium   | 9.0s  |
| flakiness/makeltFlaky.spec.ts:10                                                                     | View Trace |       |
| ✓ Intentionally flaky example › bad example: asserts cart total before the async request has settled | chromium   | 5.6s  |
| flakiness/makeltFlaky.spec.ts:10                                                                     |            |       |
| ✓ Intentionally flaky example › bad example: brittle product locator depends on card position        | chromium   | 5.8s  |
| flakiness/makeltFlaky.spec.ts:23                                                                     |            |       |
| ✓ Intentionally flaky example › bad example: brittle product locator depends on card position        | chromium   | 6.1s  |
| flakiness/makeltFlaky.spec.ts:23                                                                     |            |       |
| ✓ Intentionally flaky example › bad example: brittle product locator depends on card position        | chromium   | 7.6s  |
| flakiness/makeltFlaky.spec.ts:23                                                                     |            |       |
| ✓ Intentionally flaky example › bad example: asserts cart total before the async request has settled | chromium   | 6.8s  |
| flakiness/makeltFlaky.spec.ts:10                                                                     |            |       |
| ✓ Intentionally flaky example › bad example: brittle product locator depends on card position        | chromium   | 3.3s  |
| flakiness/makeltFlaky.spec.ts:23                                                                     |            |       |
| ✓ Intentionally flaky example › bad example: brittle product locator depends on card position        | chromium   | 4.7s  |

- HTML Report screenshot (After)

|                                                                                                      |          |      |
|------------------------------------------------------------------------------------------------------|----------|------|
| flakiness/makeltFlaky.spec.ts                                                                        |          |      |
| ✓ Intentionally flaky example › bad example: asserts cart total before the async request has settled | chromium | 5.2s |
| flakiness/makeltFlaky.spec.ts:10                                                                     |          |      |
| ✓ Intentionally flaky example › bad example: brittle product locator depends on card position        | chromium | 5.0s |
| flakiness/makeltFlaky.spec.ts:21                                                                     |          |      |

## Scenario 2 – Brittle Locator

### Issue

The locator depended on the position of a product card.

### Diagnosis

Any UI order change caused the test to fail.

### Fix

Use stable locators such as `getByRole()` or `getByTestId()`.

### Evidence

- Screenshot of failure (Before)

```

24
25 // Navigate to a product using a fragile position-based selector.
26 await page.locator(".product-grid article:nth-child(2) a").click();
27
28 // Assert that the expected product page is visible.
29 await expect(
30   page.getByRole("heading", { name: "Travel Backpack" })
31 ).toBeVisible();

```

- Screenshot after using stable locator (After)

```

24
25 // Navigate to a product using a fragile position-based selector.
26 await page.getByRole("link", { name: "View Travel Backpack" }).click();
27
28 // Assert that the expected product page is visible.
29 await expect(
30   page.getByRole("heading", { name: "Travel Backpack" })
31 ).toBeVisible();
32 });

```

```

C:\Users\299172\Documents\Saiteja_week05_gate5>npx playwright test makeItFlaky.spec.ts

Running 2 tests using 2 workers
  2 passed (10.2s)

To open last HTML report run:

  npx playwright show-report

```

---

## 5. Diagnostic Logging

### Command to run in terminal:

*npx playwright test diagnostics-logging.spec.ts*

**Test File:** diagnostics-logging.spec.ts

### Purpose

To improve debugging by capturing structured logs for every important step in the checkout flow.

### Implemented

- Correlation ID
- Structured logging
- Response time logging
- Sensitive data redaction

### Benefits

- Easy request tracing
- Faster debugging
- Secure logging without exposing sensitive information

## Evidence

- Correlation ID and Redacted log screenshot of the terminal

```
...s\diagnostics\diagnostics-logging.spec.ts:12:7 > Diagnostics and Logging > records a correlated checkout diagnostic trail
[INFO] 2026-07-04T11:27:35.007Z - test started
[INFO] 2026-07-04T11:27:35.020Z - checkout journey started {"cartSession":"w5d2-diagnostics-1783164455019"}
...sts\diagnostics\diagnostics-logging.spec.ts:113:7 > Diagnostics and Logging > redacts sensitive fields in structured logs
[INFO] 2026-07-04T11:27:35.098Z - test started
[INFO] 2026-07-04T11:27:35.103Z - payment payload prepared {"cardNumber":"[REDACTED]","headers":{"authorization":"[REDACTED]"},"orderId":"ORD-LOG-1001","token":"[REDACTED]"}
[INFO] 2026-07-04T11:27:35.123Z - test finished {"status":"passed"}
...s\diagnostics\diagnostics-logging.spec.ts:12:7 > Diagnostics and Logging > records a correlated checkout diagnostic trail
[INFO] 2026-07-04T11:27:35.132Z - cart item added {"cartSession":"w5d2-diagnostics-1783164455019","durationMs":112,"httpStatus":201,"productId":101}
[INFO] 2026-07-04T11:27:35.205Z - order placed {"cartSession":"w5d2-diagnostics-1783164455019","durationMs":24,"httpStatus":201,"orderId":5001,"orderNumber":"ORD-1009","total":4598}
[INFO] 2026-07-04T11:27:35.254Z - test finished {"status":"passed"}
2 passed (7.2s)
```

- HTML Report

|                                           |                                                                          |                      |                  |
|-------------------------------------------|--------------------------------------------------------------------------|----------------------|------------------|
| Project: chromium                         |                                                                          | 4/7/2026, 4:57:30 pm | Total time: 7.2s |
| ▼ diagnostics/diagnostics-logging.spec.ts |                                                                          |                      |                  |
| ✓                                         | Diagnostics and Logging > records a correlated checkout diagnostic trail | chromium             | 2.3s             |
|                                           | diagnostics/diagnostics-logging.spec.ts:12                               |                      |                  |
| ✓                                         | Diagnostics and Logging > redacts sensitive fields in structured logs    | chromium             | 2.3s             |
|                                           | diagnostics/diagnostics-logging.spec.ts:113                              |                      |                  |

## 6. Artifact Management

### Command to run in terminal:

*npx playwright test artifact-management.spec.ts*

*npx playwright test artifact-failure.spec.ts*

### Test Files

- artifact-management.spec.ts
- artifact-failure.spec.ts

### Purpose

To capture useful debugging evidence whenever tests pass or fail.

### Artifacts Captured

- Playwright HTML Report
- Failure Screenshot
- Cart Evidence

## Benefits

Developers can understand failures without reproducing the issue.

## Evidence

- HTML Report (Success)

**captures cart evidence without attaching it on a passing test**

artifacts/artifact-management.spec.ts:13 2.1s

chromium

✓ Run

Test Steps

Q Filter steps

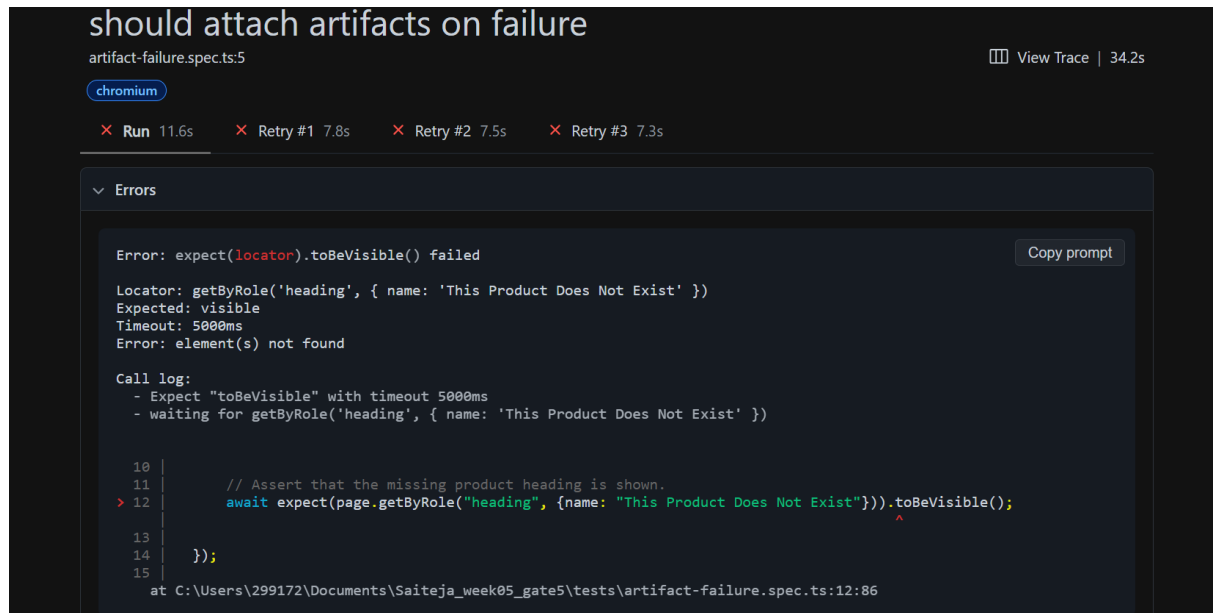
|                                                                                                                 |       |
|-----------------------------------------------------------------------------------------------------------------|-------|
| > ✓ Before Hooks                                                                                                | 1.2s  |
| > ✓ Navigate to "/debug-lab" — artifacts/artifact-management.spec.ts:17                                         | 531ms |
| > ✓ Expect "toBeVisible" getByRole('heading', { name: 'Debug Lab' }) — artifacts/artifact-management.spec.ts:20 | 95ms  |
| > ✓ POST "/api/cart/items" — artifacts/artifact-management.spec.ts:23                                           | 52ms  |
| > ✓ GET "/api/cart" — artifacts/artifact-management.spec.ts:38                                                  | 5ms   |
| > ✓ Expect "toBe" — artifacts/artifact-management.spec.ts:47                                                    | 0ms   |
| > ✓ Expect "toBe" — artifacts/artifact-management.spec.ts:48                                                    | 1ms   |
| > ✓ Expect "toMatchObject" — artifacts/artifact-management.spec.ts:51                                           | 2ms   |
| > ✓ After Hooks                                                                                                 | 683ms |

Attachments

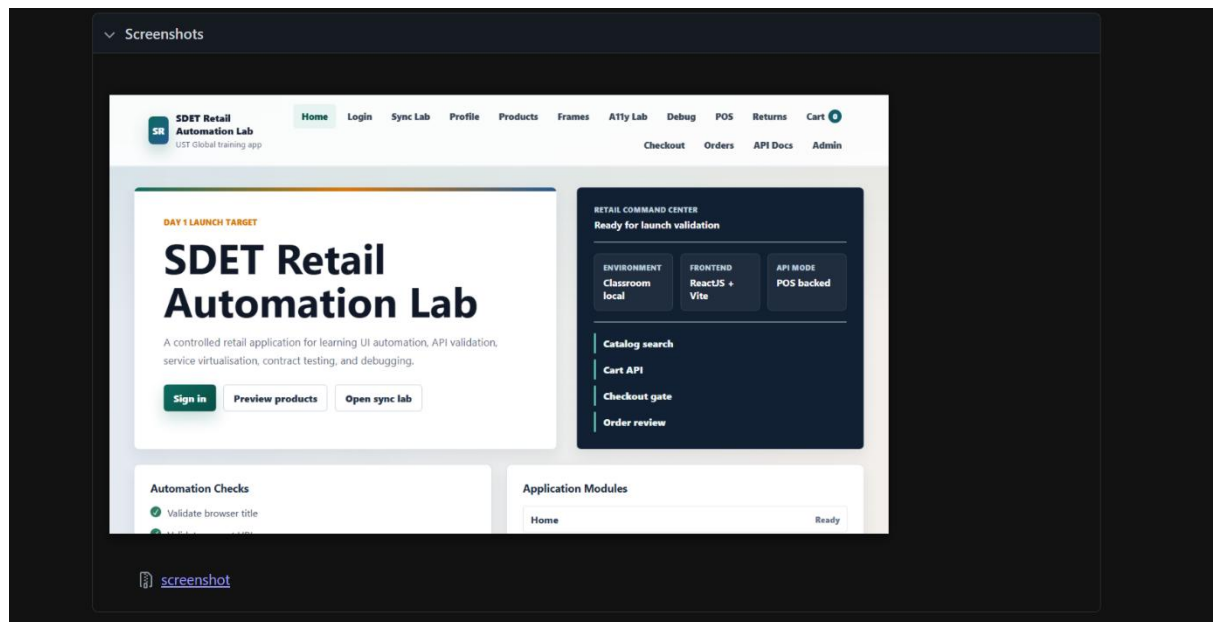
cartResponse.json

```
{
  "items": [
    {
      "id": 2,
      "userId": 1,
      "ownerKey": "user-1",
      "productId": 101,
      "quantity": 4,
      "size": "UK 9",
      "color": "Black",
      "fulfilment": "Home delivery",
      "product": {
        "id": 101,
        "name": "Running Shoes",
        "category": "Footwear",
        "price": 4499,
        "stock": 18,
        "brand": "SwiftRun",
        "sku": "FT-SHOE-101",
        "summary": "Lightweight daily trainers with breathable mesh and steady heel support.",
        "tags": [
          "Best seller",
          "Running",
          "COD eligible"
        ]
      }
    }
  ],
  "total": 17996
}
```

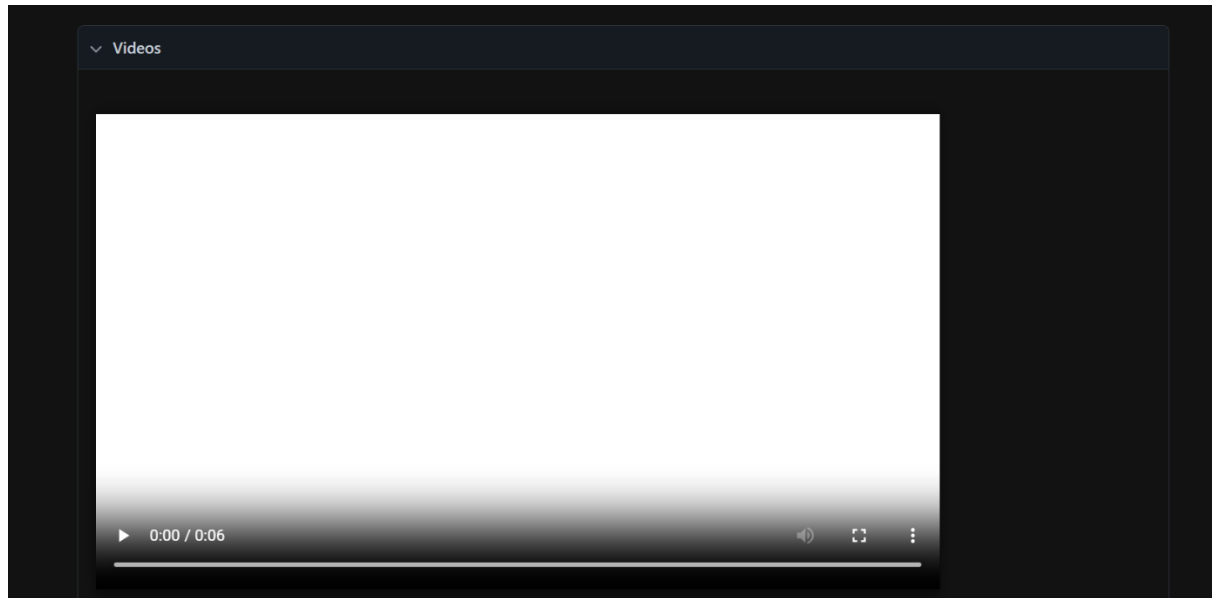
- HTML Report (Failure)



- Failure Screenshot



- Trace Back Screenshot



---

## 7. Offline & Resilient UI

### Command to run in terminal:

```
npx playwright test network-offline.spec.ts
```

**Test File:** network-offline.spec.ts

#### Scenario 1

Verify offline banner.

#### Scenario 2

Queue sales while offline.

#### Scenario 3

Synchronize queued sales after reconnecting.

#### Scenario 4

Rollback optimistic UI after failed synchronization.

#### Scenario 5

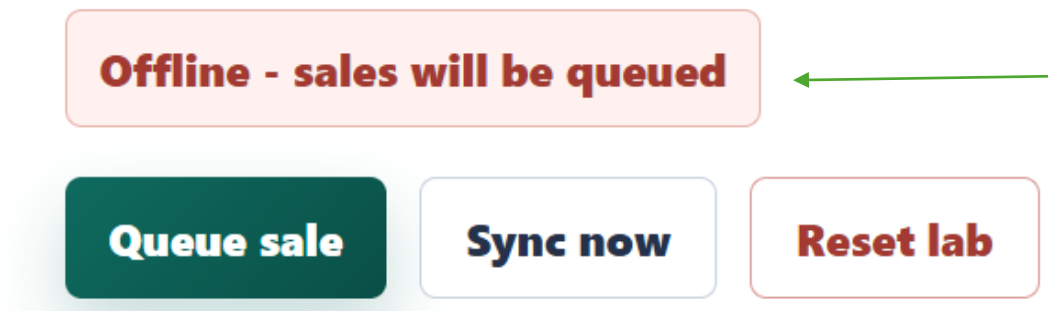
Ensure exactly-once synchronization using Idempotency.

### Benefits

The application continues functioning correctly during network interruptions and safely recovers when connectivity is restored.

### Evidence

- Offline banner screenshot



- Queue count screenshot



Pending outbox: 5



- Sync completed screenshot



Pending outbox: 0



- Rollback screenshot

**Sync failed. Optimistic sale rolled back.**



Pending outbox: **0**

**Last synced sale: SALE-9010**

- Exactly-once synchronization screenshot

Local outbox

Queued POS sales



| CLIENT SALE ID                 | PRODUCT       | STATUS | TOTAL     |
|--------------------------------|---------------|--------|-----------|
| sale-1783159737845-a45ga8e4f85 | Running Shoes | synced | Rs. 4,499 |
| sale-1783159737057-no4212gvg6  | Running Shoes | synced | Rs. 4,499 |
| sale-1783159649279-q24xywyfq59 | Running Shoes | synced | Rs. 4,499 |
| sale-1783159648748-r89dj0do947 | Running Shoes | synced | Rs. 4,499 |

---

## 8. Returns & Refund Testing

**Test File:** refund-return.spec.ts

The following business scenarios were validated.

### Partial Refund

**Command to run in terminal:**

npm playwright test partial-refund.spec.ts

Verified:

- Refund amount
- Tax calculation
- Ledger update

- UI update

## Evidence

- Before refund screenshot of UI

| Returnable Lines   |         |           |          |            |
|--------------------|---------|-----------|----------|------------|
| Refund line ledger |         |           |          |            |
| SKU                | UNIT    | PURCHASED | REFUNDED | RULE       |
| TEE                | ₹333.00 | 3         | 0        | RETURNABLE |

- After refund screenshot of UI

| Returnable Lines   |         |           |          |            |
|--------------------|---------|-----------|----------|------------|
| Refund line ledger |         |           |          |            |
| SKU                | UNIT    | PURCHASED | REFUNDED | RULE       |
| TEE                | ₹333.00 | 3         | 1        | RETURNABLE |

- Terminal Output

```
Running 1 test using 1 worker
[chromium] > tests\refunds\partial-refund.spec.ts:13:7 > Partial Refund > should successfully process a partial refund
[INFO] 2026-07-04T10:58:20.250Z - test started
[INFO] 2026-07-04T10:58:20.258Z - Seeding order
[INFO] 2026-07-04T10:58:20.391Z - Opening Returns page {"orderId":7020}
[INFO] 2026-07-04T10:58:21.378Z - Submitting partial refund
[INFO] 2026-07-04T10:58:21.441Z - Ledger verified {"status":"PARTIALLY_REFUNDED","refundCount":1}
[INFO] 2026-07-04T10:58:21.691Z - test finished {"status":"passed"}
  1 passed (8.2s)

To open last HTML report run:
  npx playwright show-report

C:\Users\299172\Documents\Saiteja_week05_gate5>
```

- HTML Report

Partial Refund

## should successfully process a partial refund

refunds/partial-refund.spec.ts:13 3.3s

chromium

✓ Run

Test Steps

Filter steps

|                                                                                           |       |
|-------------------------------------------------------------------------------------------|-------|
| > ✓ Before Hooks                                                                          | 1.6s  |
| > ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14                            | 101ms |
| > ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32                                      | 1ms   |
| > ✓ Navigate to "/returns?orderId=7020" — ./src/api/refund.api.ts:107                     | 678ms |
| > ✓ Expect "toContainText" getByTestId('refund-line') — refunds/partial-refund.spec.ts:32 | 222ms |
| > ✓ Expect "toHaveText" getByTestId('refunded-TEE') — refunds/partial-refund.spec.ts:33   | 65ms  |
| > ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43                                      | 10ms  |
| > ✓ Expect "toBeTruthy" — refunds/partial-refund.spec.ts:44                               | 1ms   |

- Evidence

Attachments

order.json

```
{
  "id": 7020,
  "orderNumber": "RET-7020",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 0,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 104895,
  "refunds": [],
  "refundCount": 0,
  "lastRefund": null,
  "status": "PAID"
}
```

> refund.json

> ledger.json

## Full Refund

### Command to run in terminal:

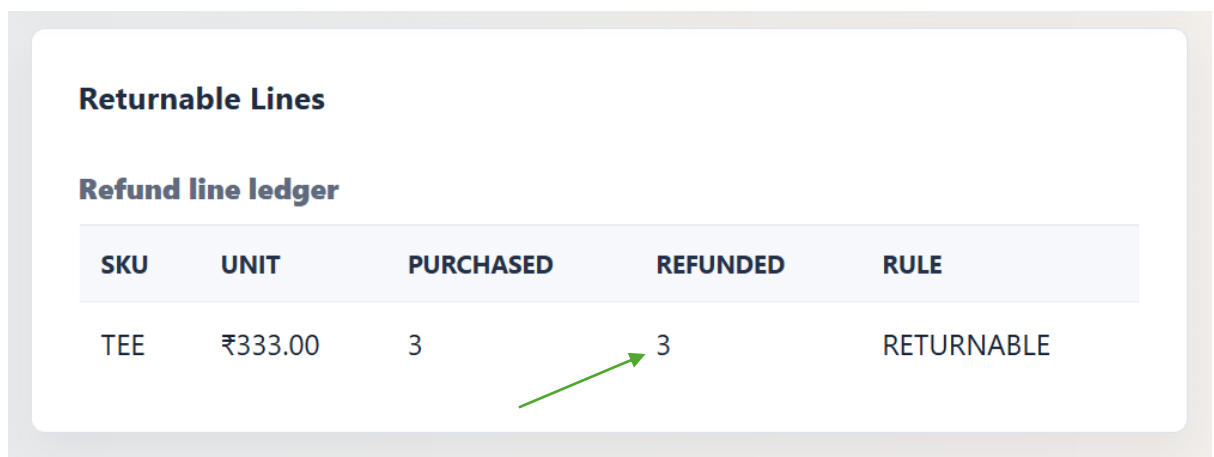
```
npx playwright test full-refund.spec.ts
```

Verified:

- Complete refund
- Refundable balance becomes zero
- Ledger updated correctly

### Evidence

- Full refund screenshot of UI



| Returnable Lines   |         |           |          |            |
|--------------------|---------|-----------|----------|------------|
| Refund line ledger |         |           |          |            |
| SKU                | UNIT    | PURCHASED | REFUNDED | RULE       |
| TEE                | ₹333.00 | 3         | 3        | RETURNABLE |

- Terminal Output

```
Running 1 test using 1 worker
[chromium] > tests\refunds\full-refund.spec.ts:13:7 > Full Refund > should successfully process a full refund
[INFO] 2026-07-04T11:03:36.847Z - test started
[INFO] 2026-07-04T11:03:36.856Z - Creating order
[INFO] 2026-07-04T11:03:37.020Z - Refunding entire order
[INFO] 2026-07-04T11:03:37.080Z - Full refund completed {"status":"REFUNDED"}
[INFO] 2026-07-04T11:03:38.296Z - test finished {"status":"passed"}
1 passed (7.3s)
```

To open last HTML report run:

```
npx playwright show-report
```

```
C:\Users\299172\Documents\Saiteja_week05_gate5>
```

- HTML Report

Full Refund

## should successfully process a full refund

refunds/full-refund.spec.ts:13 3.2s

chromium

✓ Run

Test Steps

Filter steps

|                                                                    |       |
|--------------------------------------------------------------------|-------|
| > ✓ Before Hooks                                                   | 1.6s  |
| > ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14     | 120ms |
| > ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32               | 4ms   |
| > ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43               | 14ms  |
| > ✓ Expect "toBeTruthy" — refunds/full-refund.spec.ts:36           | 1ms   |
| > ✓ Expect "toBe" — refunds/full-refund.spec.ts:42                 | 1ms   |
| > ✓ GET "/api/refund-lab/orders/7021" — ./src/api/refund.api.ts:91 | 8ms   |
| > ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:98               | 0ms   |
| > ✓ Expect "toBe" — refunds/full-refund.spec.ts:53                 | 0ms   |

- Evidence

Attachments

order.json

```
{  "id": 7021,  "orderNumber": "RET-7021",  "ownerKey": "user-1",  "placedOn": "2026-06-29",  "daysAgo": 5,  "returnWindowDays": 30,  "paymentMethod": "Original card",  "lines": [    {      "sku": "TEE",      "name": "Training Tee",      "unitPaise": 33300,      "qty": 3,      "refundedQty": 0,      "finalSale": false,      "nonReturnable": false    }  ],  "lineTotalPaise": 99900,  "taxPaise": 4995,  "totalPaise": 104895,  "refundableBalancePaise": 104895,  "refunds": [],  "refundCount": 0,  "lastRefund": null,  "status": "PAID"}
```

> refund.json

> ledger.json

## Refund Eligibility

### Command to run in terminal:

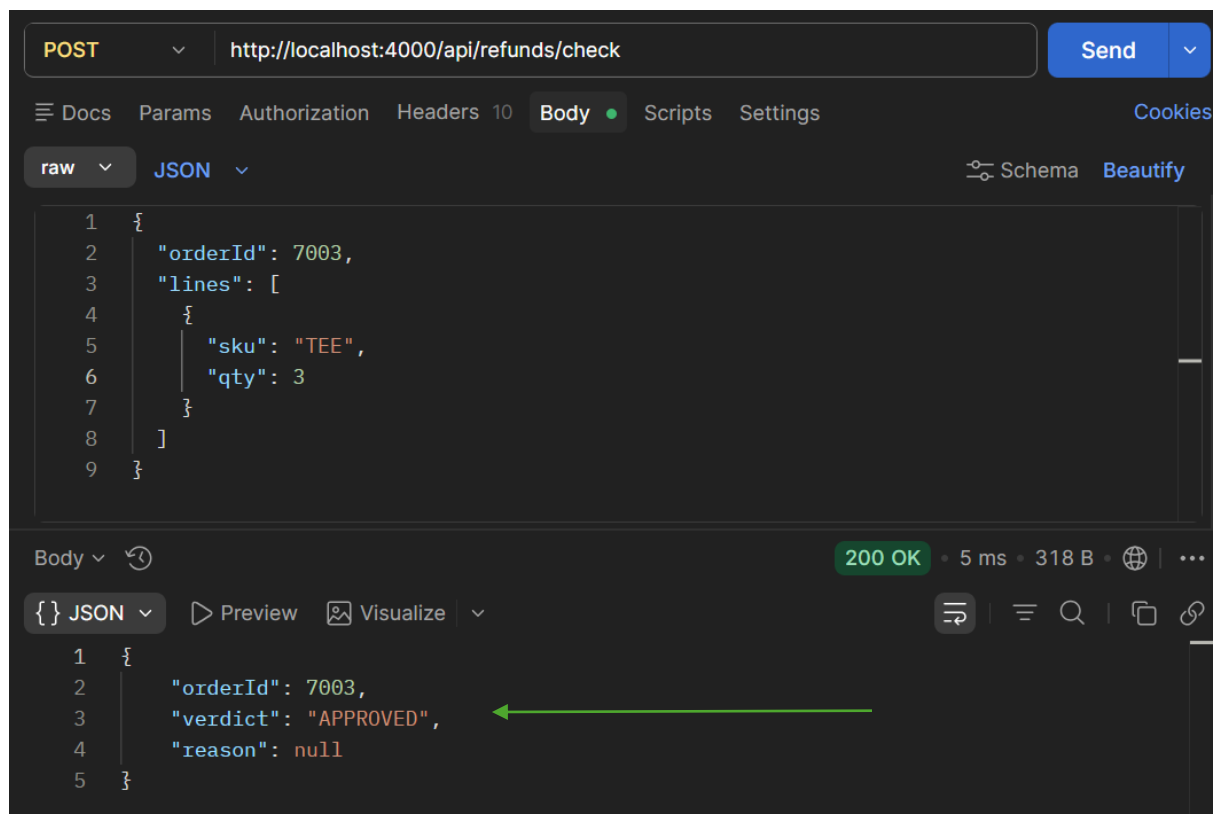
`npx playwright test refund-eligibility.spec.ts`

Verified:

- Valid refund

### Evidence

- Refund eligibility screenshot in postman



- Terminal Output

```

[INFO] 2026-07-04T11:05:23.837Z - Refund eligibility verified {"verdict":"OVER_REFUND"}
[INFO] 2026-07-04T11:05:23.870Z - test finished {"status":"passed"}
[chromium] > tests\refunds\refund-eligibility.spec.ts:27:9 > Refund Eligibility > Valid Refund
[INFO] 2026-07-04T11:05:23.961Z - test started
[INFO] 2026-07-04T11:05:23.966Z - Creating order
[INFO] 2026-07-04T11:05:24.056Z - Checking refund eligibility
[INFO] 2026-07-04T11:05:24.089Z - Refund eligibility verified {"verdict":"APPROVED"}
[INFO] 2026-07-04T11:05:24.122Z - test finished {"status":"passed"}
  2 passed (6.5s)

```

To open last HTML report run:

```
npx playwright show-report
```

C:\Users\299172\Documents\Saiteja\_week05\_gate5>

- HTML Report

Refund Eligibility
next »

# Valid Refund

refunds/refund-eligibility.spec.ts:27

2.4s

chromium

✓ Run

Test Steps

Filter steps

Before Hooks
2.2s

POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14
65ms

Expect "toBeTruthy" — ./src/api/refund.api.ts:32
1ms

POST "/api/refunds/check" — ./src/api/refund.api.ts:66
9ms

Expect "toBeTruthy" — ./src/api/refund.api.ts:82
0ms

Expect "toBe" — refunds/refund-eligibility.spec.ts:52
1ms

After Hooks
936ms

- Evidence

Attachments

order.json

```

{
  "id": 7023,
  "orderNumber": "RET-7023",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 0,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 104895,
  "refunds": [],
  "refundCount": 0,
  "lastRefund": null,
  "status": "PAID"
}

```

refundCheck.json

---

## Rejected Refund

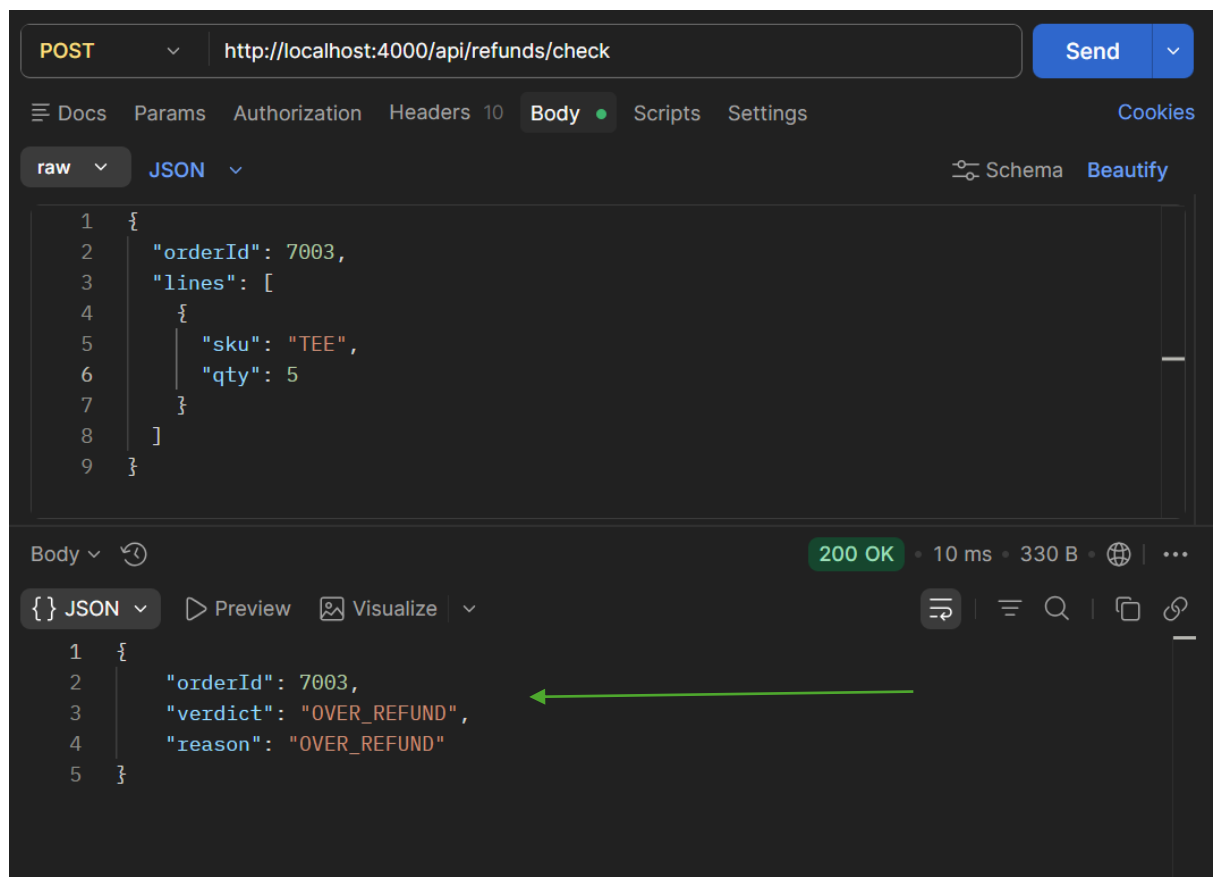
### Command to run in terminal:

`npx playwright test rejected-refund.spec.ts`

Verified that over-refund requests are rejected.

### Evidence

- Rejected refund screenshot from postman



- Terminal Output

```
Running 1 test using 1 worker
[chromium] > tests\refunds\rejected-refund.spec.ts:11:7 > Rejected Refund > should reject an over-refund request
[INFO] 2026-07-04T11:11:33.337Z - test started
[INFO] 2026-07-04T11:11:33.346Z - Creating order
[INFO] 2026-07-04T11:11:33.520Z - Attempting over refund
[INFO] 2026-07-04T11:11:33.553Z - Over refund rejected {"reason":"OVER_REFUND"}
[INFO] 2026-07-04T11:11:33.591Z - Verified ledger remains unchanged after rejection {"refundableBalancePaise":104895}
[INFO] 2026-07-04T11:11:33.680Z - test finished {"status":"passed"}
1 passed (8.6s)

To open last HTML report run:

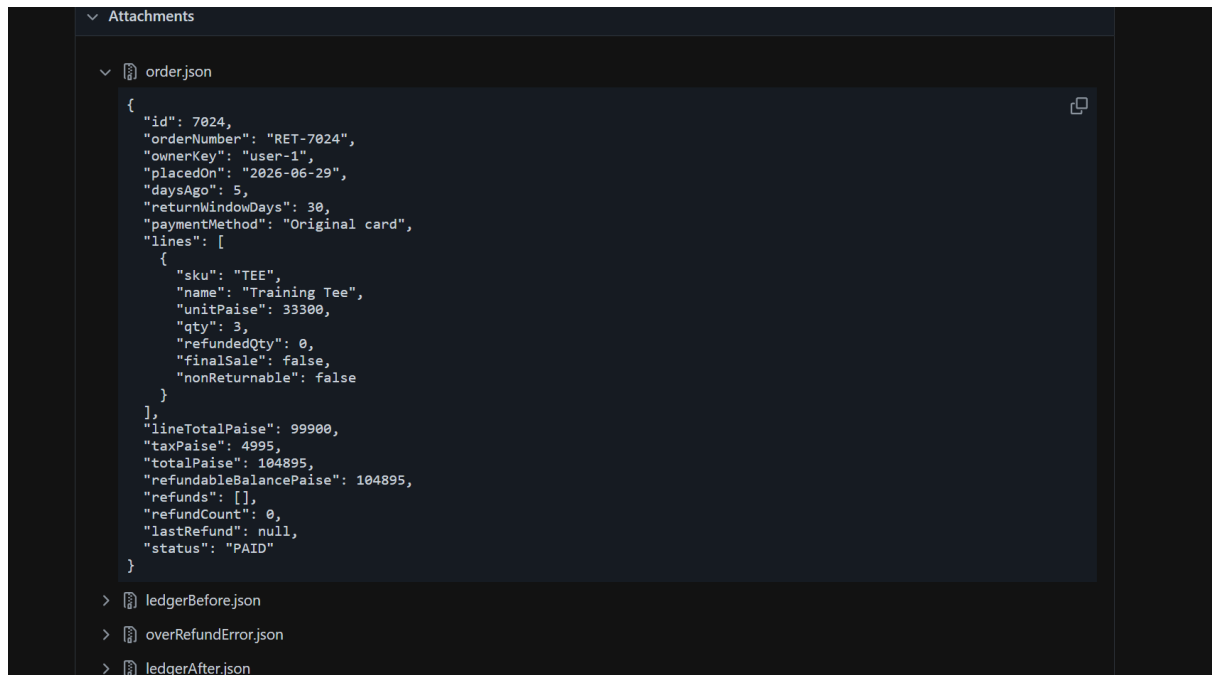
npx playwright show-report
```

- HTML Report

The screenshot shows a detailed HTML report for a failed test. At the top, the test title is 'should reject an over-refund request' from the file 'refunds/rejected-refund.spec.ts:11', with a duration of 1.9s. Below the title, there's a 'chromium' browser icon and a 'Run' button. The main section is titled 'Test Steps' and contains a list of steps with their durations. The steps are: 'Before Hooks' (1.8s), 'POST "/api/refund-lab/orders"' (108ms), 'Expect "toBeTruthy"' (2ms), 'GET "/api/refund-lab/orders/7024"' (11ms), 'Expect "toBeTruthy"' (0ms), 'POST "/api/refunds"' (11ms), 'Expect "toBe"' (2ms), 'Expect "toBe"' (1ms), and 'Expect "toBe"' (1ms). The report is styled with a dark theme and green checkmarks for successful steps.

| Step                                                               | Duration |
|--------------------------------------------------------------------|----------|
| > ✓ Before Hooks                                                   | 1.8s     |
| > ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14     | 108ms    |
| > ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32               | 2ms      |
| > ✓ GET "/api/refund-lab/orders/7024" — ./src/api/refund.api.ts:91 | 11ms     |
| > ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:98               | 0ms      |
| > ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43               | 11ms     |
| > ✓ Expect "toBe" — refunds/rejected-refund.spec.ts:39             | 2ms      |
| > ✓ Expect "toBe" — refunds/rejected-refund.spec.ts:49             | 1ms      |
| > ✓ Expect "toBe" — refunds/rejected-refund.spec.ts:50             | 1ms      |

- Evidence



---

## Duplicate Refund

### Command to run in terminal:

*npx playwright test duplicate-refund-idempotency.spec.ts*

The same refund request was submitted twice.

The application returned the existing refund instead of creating another one.

### Evidence

- Duplicate refund
- Screenshot of First time request(Status Code = 201 Created) from Postman

POST http://localhost:4000/api/refunds Send

Docs Params Authorization Headers 11 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "orderId": 7001,
3   "lines": [
4     {
5       "sku": "TEE",
6       "qty": 2
7     }
8   ]
9 }
```

Body JSON Preview Visualize

201 Created • 10 ms • 638 B •

```
1 {
2   "refundId": 8002,
3   "orderId": 7001,
4   "ownerKey": "user-1",
5   "idempotencyKey": "refund-2026-002",
6   "amountPaise": 69930,
7   "lineAmountPaise": 66600,
8   "taxPaise": 3330,
9   "taxShares": [
10    1665,
11    1665,
12    1665
13  ]
14 }
```

- Terminal Output

```
Running 1 test using 1 worker
...efund-idempotency.spec.ts:12:7 › Duplicate Refund and Idempotency › should process only one refund for duplicate requests
[INFO] 2026-07-04T11:17:13.756Z - test started
[INFO] 2026-07-04T11:17:13.760Z - Creating order
[INFO] 2026-07-04T11:17:13.854Z - Submitting first refund request
[INFO] 2026-07-04T11:17:13.870Z - Submitting duplicate refund request
[INFO] 2026-07-04T11:17:13.901Z - Verified idempotent refund {"refundCount":1,"refundableBalancePaise":69930}
[INFO] 2026-07-04T11:17:13.986Z - test finished {"status":"passed"}
1 passed (5.6s)

To open last HTML report run:

npx playwright show-report
```

- HTML Report

Duplicate Refund and Idempotency

should process only one refund for duplicate requests

refunds/duplicate-refund-idempotency.spec.ts:121.6s

chromium

✓ Run

Test Steps

Q Filter steps

> ✓ Before Hooks1.4s

> ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:1477ms

> ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:321ms

> ✓ POST "/api/refunds" — ./src/api/refund.api.ts:438ms

> ✓ Expect "toBeTruthy" — refunds/duplicate-refund-idempotency.spec.ts:380ms

> ✓ POST "/api/refunds" — ./src/api/refund.api.ts:438ms

> ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:531ms

> ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:590ms

> ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:600ms

- Evidence



- Screenshot of Second time request(Status Code = 200 OK) from Postman

POST http://localhost:4000/api/refunds Send

Docs Params Authorization Headers 11 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "orderId": 7001,
3   "lines": [
4     {
5       "sku": "TEE",
6       "qty": 2
7     }
8   ]
9 }
```

Body Body 200 OK • 11 ms • 552 B • ...

{ } JSON Preview Visualize

```
1 {
2   "refundId": 8002,
3   "orderId": 7001,
4   "ownerKey": "user-1",
5   "idempotencyKey": "refund-2026-002",
6   "amountPaise": 69930,
7   "lineAmountPaise": 66600,
8   "taxPaise": 3330,
9   "taxShares": [
10    1665,
11    1665,
12    1665
13  ]
14 }
```

- Terminal Output

```
Running 1 test using 1 worker
...efund-idempotency.spec.ts:12:7 > Duplicate Refund and Idempotency > should process only one refund for duplicate requests
[INFO] 2026-07-04T11:17:13.756Z - test started
[INFO] 2026-07-04T11:17:13.760Z - Creating order
[INFO] 2026-07-04T11:17:13.854Z - Submitting first refund request
[INFO] 2026-07-04T11:17:13.870Z - Submitting duplicate refund request
[INFO] 2026-07-04T11:17:13.901Z - Verified idempotent refund {"refundCount":1,"refundableBalancePaise":69930}
[INFO] 2026-07-04T11:17:13.986Z - test finished {"status":"passed"}
1 passed (5.6s)

To open last HTML report run:

npx playwright show-report
```

- HTML Report

Duplicate Refund and Idempotency

## should process only one refund for duplicate requests

refunds/duplicate-refund-idempotency.spec.ts:12 1.6s

chromium

✓ Run

Test Steps

Filter steps

|                                                                           |      |
|---------------------------------------------------------------------------|------|
| > ✓ Before Hooks                                                          | 1.4s |
| > ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14            | 77ms |
| > ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32                      | 1ms  |
| > ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43                      | 8ms  |
| > ✓ Expect "toBeTruthy" — refunds/duplicate-refund-idempotency.spec.ts:38 | 0ms  |
| > ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43                      | 8ms  |
| > ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:53       | 1ms  |
| > ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:59       | 0ms  |
| > ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:60       | 0ms  |

- Evidence

Attachments

order.json

```
{
  "id": 7027,
  "orderNumber": "RET-7027",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 0,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 104895,
  "refunds": [],
  "refundCount": 0,
  "lastRefund": null,
  "status": "PAID"
}
```

> refund.json

> replayRefund.json

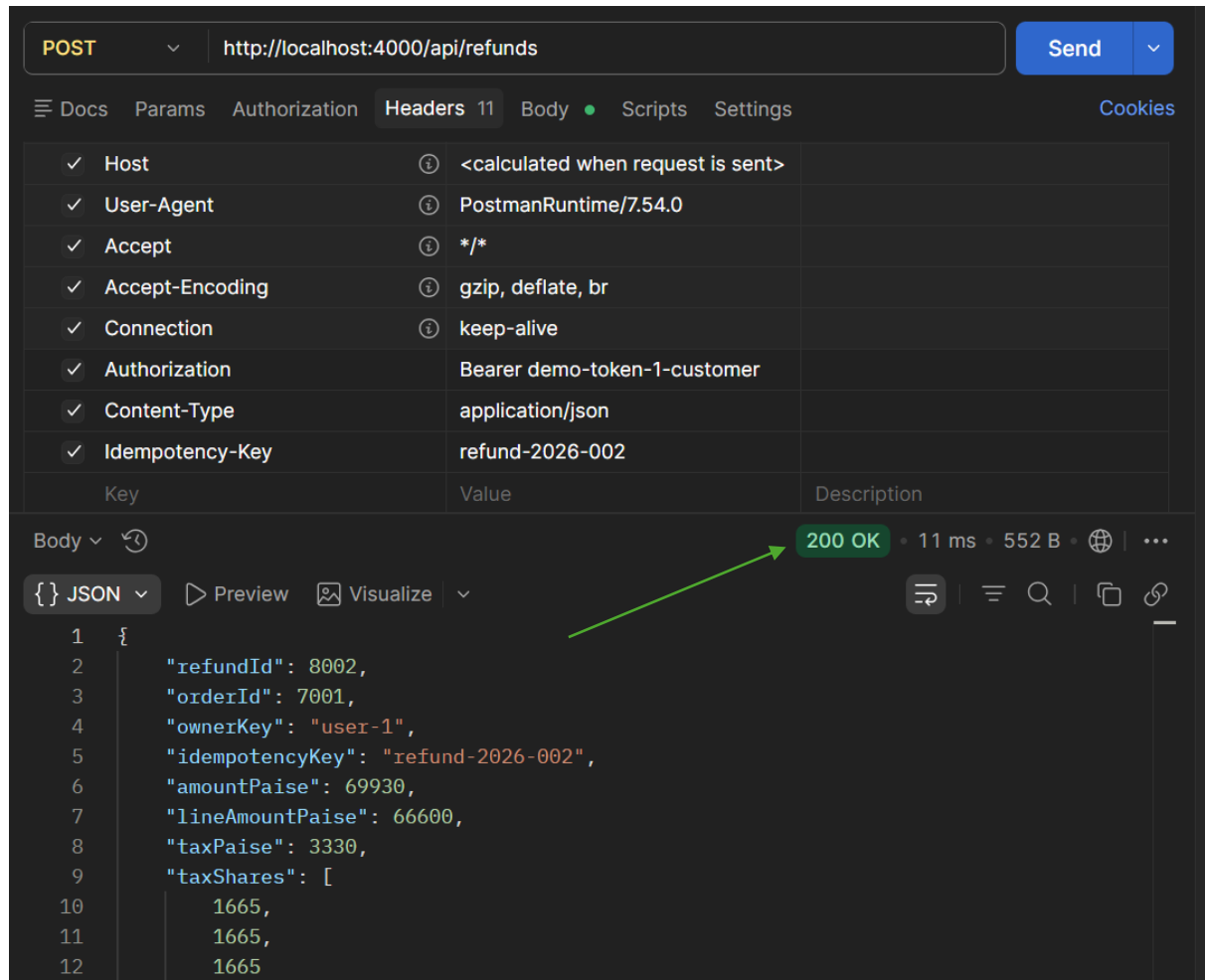
> ledger.json

## Idempotency

Verified that duplicate requests with the same idempotency key are processed only once.

## Evidence

- Idempotency screenshot



- Terminal Output

```
Running 1 test using 1 worker
...efund-idempotency.spec.ts:12:7 > Duplicate Refund and Idempotency > should process only one refund for duplicate requests
[INFO] 2026-07-04T11:17:13.756Z - test started
[INFO] 2026-07-04T11:17:13.760Z - Creating order
[INFO] 2026-07-04T11:17:13.854Z - Submitting first refund request
[INFO] 2026-07-04T11:17:13.870Z - Submitting duplicate refund request
[INFO] 2026-07-04T11:17:13.901Z - Verified idempotent refund {"refundCount":1,"refundableBalancePaise":69930}
[INFO] 2026-07-04T11:17:13.986Z - test finished {"status":"passed"}
1 passed (5.6s)

To open last HTML report run:

npx playwright show-report
```

- HTML Report

Duplicate Refund and Idempotency

## should process only one refund for duplicate requests

refunds/duplicate-refund-idempotency.spec.ts:12 1.6s

chromium

✓ Run

Test Steps

Q Filter steps

|                                                                           |      |
|---------------------------------------------------------------------------|------|
| > ✓ Before Hooks                                                          | 1.4s |
| > ✓ POST "/api/refund-lab/orders" — ./src/api/refund.api.ts:14            | 77ms |
| > ✓ Expect "toBeTruthy" — ./src/api/refund.api.ts:32                      | 1ms  |
| > ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43                      | 8ms  |
| > ✓ Expect "toBeTruthy" — refunds/duplicate-refund-idempotency.spec.ts:38 | 0ms  |
| > ✓ POST "/api/refunds" — ./src/api/refund.api.ts:43                      | 8ms  |
| > ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:53       | 1ms  |
| > ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:59       | 0ms  |
| > ✓ Expect "toBe" — refunds/duplicate-refund-idempotency.spec.ts:60       | 0ms  |

- Evidence

Attachments

order.json

```
{
  "id": 7027,
  "orderNumber": "RET-7027",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPaise": 33300,
      "qty": 3,
      "refundedQty": 0,
      "finalSale": false,
      "nonReturnable": false
    }
  ],
  "lineTotalPaise": 99900,
  "taxPaise": 4995,
  "totalPaise": 104895,
  "refundableBalancePaise": 104895,
  "refunds": [],
  "refundCount": 0,
  "lastRefund": null,
  "status": "PAID"
}
```

> refund.json

> replayRefund.json

> ledger.json

## Money Precision

Verified all monetary calculations using **paise** instead of floating-point values.

Validated:

- Refund Amount

- Tax
- Ledger Balance

## Evidence

- Money precision screenshot

```
"amountPaise": 69930,
"lineAmountPaise": 66600,
"taxPaise": 3330,
"taxShares": [
  1665,
  1665,
  1665
],
```

## 9. Failure Analysis

Every issue in this assessment follows the same debugging approach.

### Symptom

Identify the observed failure.

### Diagnosis

Analyze logs, assertions and application behavior to determine the root cause.

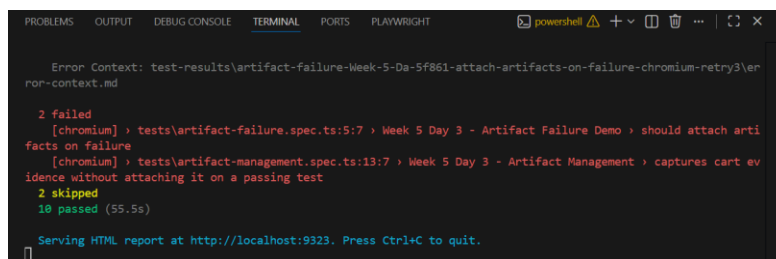
### Fix

Apply the appropriate solution instead of temporary workarounds.

## Evidence

Validate the fix using:

- Screenshot



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PLAYWRIGHT
Error Context: test-results\artifact-failure-Week-5-Da-5f861-attach-artifacts-on-failure-chromium-retry3\error-context.md

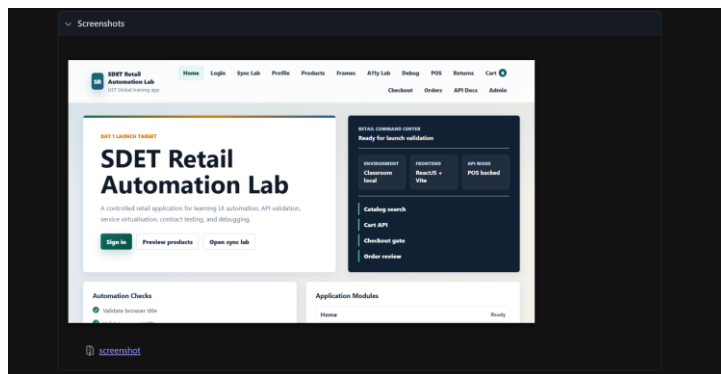
2 failed
[chromium] › tests\artifact-failure.spec.ts:5:7 › Week 5 Day 3 - Artifact Failure Demo › should attach artifacts on failure
[chromium] › tests\artifact-management.spec.ts:13:7 › Week 5 Day 3 - Artifact Management › captures cart evidence without attaching it on a passing test
2 skipped
10 passed (55.5s)
Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.

```

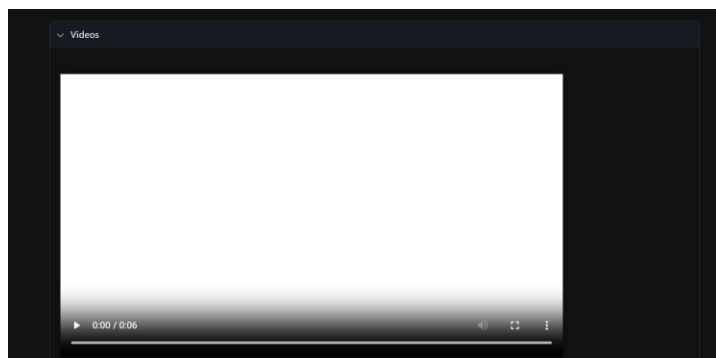
- HTML Report



- Failure Screenshot



- Trace Back Video



## 10. Debugging Maturity

This assessment demonstrates debugging maturity through:

- Identifying flaky tests
- Diagnosing root causes
- Structured diagnostic logging
- Artifact collection
- Offline resilience testing

- Business rule validation
  - Refund validation
  - Idempotency testing
  - Financial validation using paise
- 

**11. Challenges Faced**

- Diagnosing flaky tests
  - Handling offline scenarios
  - Validating idempotent requests
  - Preventing duplicate refunds
  - Testing financial calculations
  - Capturing useful debugging evidence
- 

**12. Conclusion**

This assessment successfully demonstrates stable Playwright automation by focusing on diagnosing issues, implementing reliable solutions, collecting meaningful evidence, and clearly explaining the results. The project validates critical business scenarios such as offline operations, refund processing, artifact management, and structured diagnostic logging while improving overall debugging maturity.

---

**13. Evidence Summary**

| Assessment Area     | Evidence to Include                                                               |
|---------------------|-----------------------------------------------------------------------------------|
| Flakiness Analysis  | Before failure screenshot, After fix screenshot, HTML Report                      |
| Diagnostic Logging  | Correlation ID logs, Structured logs, Redacted logs, Trace flow screenshot        |
| Artifact Management | HTML Report (Pass), HTML Report (Fail), Failure Screenshot, Trace Back Screenshot |

| Assessment Area          | Evidence to Include                                                                          |
|--------------------------|----------------------------------------------------------------------------------------------|
| Offline & Resilient UI   | Offline banner, Queue count, Rollback, Sync success                                          |
| Returns & Refund Testing | Partial refund, Full refund, Rejected refund, Duplicate refund, Idempotency, Money precision |